Classic + Simple

Code guide for `TeleCon_Classic_Simple`

ESP32 DevKit · Bluetooth Classic · text lines

Version 1.0 2026

Abstract

This note is the Arduino sketch `ESP32_noCam/Classic_Simple/TeleCon_Classic_Simple/`. The phone setting is Classic + Simple. The board name is `ESP32-TC-RC-BT-Simple`. The handshake is `RC:CONNECT,proto,simple`. The pages below show every file the IDE compiles, how one session runs, and the three places a vehicle sketch is supposed to change: `onControl()`, `onButton()`, and `kTelemetryChannels[]`.

Change this Already working Leave it alone

Contents

1 The folder Arduino opens **2**

2 One session, from pair to plot **2**

3 Where to change the sketch **3**

4 Name, pin, and switches **3**

5 What the sketch remembers **4**

6 CH1 to CH8 **5**

7 Steering servo **7**

8 Your actuators **9**

9 setup **10**

10 loop **11**

11 Handshake and the text parser **12**

12 Serial inject and the monitor **14**

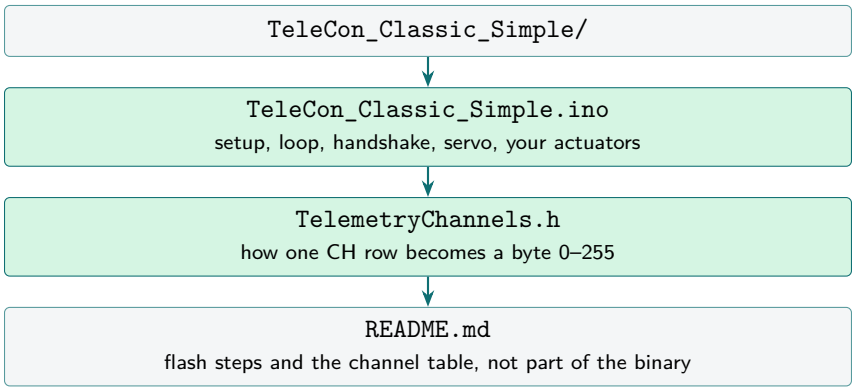
13 A short checklist before you flash **16**

A Full `TeleCon_Classic_Simple.ino` **16**

B Full `TelemetryChannels.h` **23**

1 The folder Arduino opens

Open the folder that contains the `.ino`. The IDE compiles every source in that folder. `README.md` is not compiled.

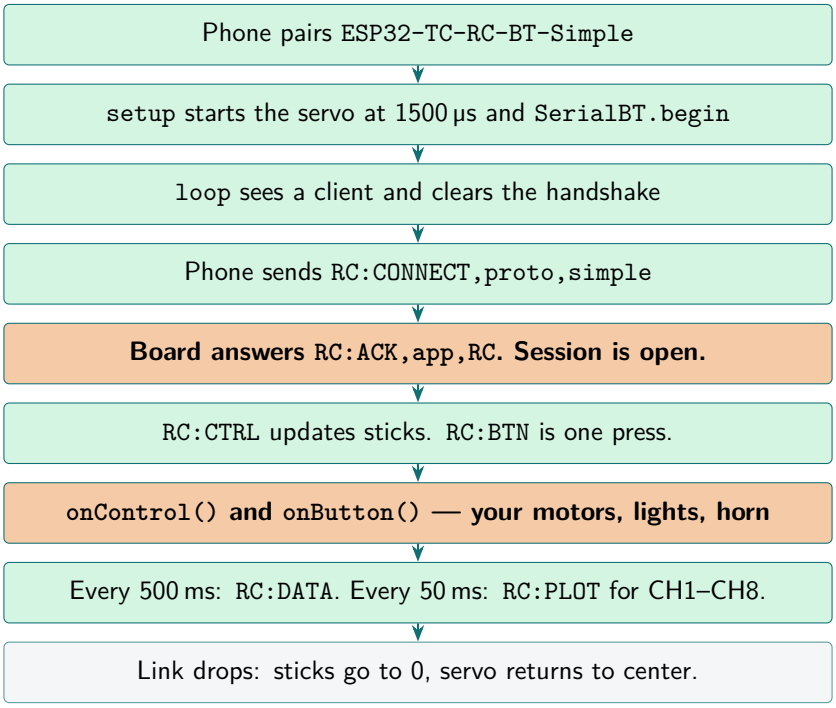


Board in the Arduino IDE: **ESP32 Dev Module**. Serial Monitor: 115200 baud, line ending Newline. In the phone: Board DevKit, connection Classic + Simple. Pair `ESP32-TC-RC-BT-Simple`. If Android asks for a PIN, it is `1234`.

Do not copy this sketch onto an ESP32-CAM. Video uses the video-only sketch on a second board. Sticks stay on this Bluetooth link.

2 One session, from pair to plot

Nothing in `onControl()` runs until the phone has completed the handshake. Telemetry is the same: `loop()` returns early while `handshakeOk` is false.



3 Where to change the sketch

Most of the file is the link. A vehicle project only needs the orange rows. Leave the parser, the Bluetooth name, and `proto,simple` alone unless you are deliberately making a different product. The phone matches this sketch by that name and that handshake.

You want	Edit	Leave
Drive motors, a second servo, lights	`onControl()`, the block marked `YOUR CODE HERE`	`handleLine()`, `getValue()`
Horn, pan-center, extra buttons	`onButton()`	Button ids the phone already sends
Stop motors when Bluetooth drops	`applyFailSafe()`, the second `YOUR CODE HERE`	The stick reset above that comment
What CH1–CH8 show on the phone	`kTelemetryChannels[]` in the `.ino`	Do not add a ninth row
Real temperature or battery	`telemetryVehicle()`, cases `TEL_TEMP` and `TEL_BATT`, or change that row to `TEL_ADC`	The 0–255 range
Servo on another pin, or no servo	`PIN_STEER_SERVO`, or `ENABLE_STEER_SERVO` set to 0	`rx = 0` means 1500 µs
Quiet Serial Monitor	`ENABLE_DEBUG`, `ENABLE_PLOT`, `ENABLE_SERIAL_INJECT` set to 0	Flash again after the change
A different phone protocol	Do not retarget this file	Use Classic Binary or BLE Binary

`rx` is already shifted by the phone trim before it arrives. This sketch does not store a steering center. Do not add a `RC:SET,steer_center` handler.

4 Name, pin, and switches

```
// === CONFIG ===
#define BT_NAME             "ESP32-TC-RC-BT-Simple"
#define BT_PIN              "1234"
#define APP_PREFIX          "RC"
#define PROTO_WIRE          "simple"
#define SERIAL_BAUD         115200

// 1 = Serial banners + stick/HS/IO logs. Set 0 when Connect works.
#define ENABLE_DEBUG        1

#if ENABLE_DEBUG
  #define DBG_PRINT(x)      Serial.print(x)
  #define DBG_PRINTLN(x)    Serial.println(x)
  #define DBG_PRINTF(...)   Serial.printf(__VA_ARGS__)
#else
  #define DBG_PRINT(x)      ((void)0)
  #define DBG_PRINTLN(x)    ((void)0)
  #define DBG_PRINTF(...)   ((void)0)
#endif

#define TELEMETRY_DATA_MS   500
#define ENABLE_PLOT         1
#define PLOT_MS             50

#define ENABLE_SERIAL_INJECT 1
#define SERIAL_MAX_LINE     192
```

```
42
43  // Demo HUD values (replace with ADC reads when you wire sensors)
44  #define DEMO_MOTOR_TEMP     90
45  #define DEMO_BATT_PCT       76
```

`BT_NAME` and `BT_PIN` are what Android sees while pairing. `APP_PREFIX` and `PROTO_WIRE` are what `handleConnect()` accepts. If either string changes, the phone setting Classic + Simple will get `RC:NAK` and `onControl()` will never run.

`ENABLE_DEBUG` turns the `DBG_*` macros into `Serial.print`, or into nothing. The session still runs when they are empty. `ENABLE_PLOT` is the 50 ms `RC:PLOT` stream. `ENABLE_SERIAL_INJECT` lets you paste one `RC:DATA` or `RC:PLOT` line into the Serial Monitor after the handshake; the next automatic packet replaces it. `DEMO_MOTOR_TEMP` and `DEMO_BATT_PCT` are the stand-in HUD numbers until a sensor is wired. They are not read from a pin.

5 What the sketch remembers

```
47  // === BLUETOOTH ===
48  BluetoothSerial SerialBT;
49
50  bool handshakeOk = false;
51  bool btWasConnected = false;
52
53  int g_lx = 0, g_ly = 0, g_rx = 0, g_ry = 0;
54  int g_lk = 512, g_rk = 512;
55  uint8_t g_sw = 0;
56
57  unsigned long lastDataMs = 0;
58  unsigned long lastPlotMs = 0;
59
60  #if ENABLE_SERIAL_INJECT
61  static String serialLineBuf;
62  #endif
63
64  static uint16_t lastPrintLX = 0xFFFF;
65  static uint16_t lastPrintLY = 0xFFFF;
66  static uint16_t lastPrintRX = 0xFFFF;
67  static uint16_t lastPrintRY = 0xFFFF;
68  static uint16_t lastPrintKnobL = 0xFFFF;
69  static uint16_t lastPrintKnobR = 0xFFFF;
70  static uint8_t lastPrintSw = 0xFF;
```

One `RC:CTRL` line fills the six controls and the switch byte. Until the first line arrives, sticks are 0 and both knobs are 512 (mid-scale of 0–1023). `handshakeOk` is the gate in `loop()` and in `handleLine()`. `btWasConnected` exists only so a connect or a drop is noticed once, not on every pass of `loop()`.

The `lastPrint*` variables belong to the Serial Monitor. They are not sent to the phone. A stick is printed only when its 12-bit image changes, so a resting stick does not flood the monitor.

Name	Range	Phone control
`g_lx`, `g_ly`	−100 … 100	Left stick X, left stick Y (throttle)
`g_rx`, `g_ry`	−100 … 100	Right stick X (steering), right stick Y
`g_lk`, `g_rk`	0 … 1023	Left knob, right knob. 512 is the middle.
`g_sw`	8 bits, sent as hex	Bit 0 is the lights switch. Bits 1–5 are the other switches.

6 CH1 to CH8

The phone has eight plot channels and no ninth. Row 0 of the table is CH1, which the wire calls `v0`. Row 7 is CH8, `v7`. Each sample is one byte, 0–255.

```
// Phone CH1...CH8. Change a row to TEL_ADC, TEL_GPIO, or TEL_PWM and set the pin.
// Examples:
//   { TEL_ADC,  TEL_TEMP,  34, -1, 0, 1 },            // analog pin 34
//   { TEL_GPIO, TEL_LIGHTS, 15, -1, 0, 1 },           // digital pin 15
//   { TEL_PWM,  TEL_THROTTLE, 25, 0, 0, 255 },        // 8-bit motor PWM on GPIO 25
static const TelChannel kTelemetryChannels[8] = {
  { TEL_VEHICLE, TEL_TEMP,     -1,              -1,   0,   1 },
  { TEL_VEHICLE, TEL_THROTTLE, -1,              -1,   0,   1 },
  { TEL_VEHICLE, TEL_BATT,     -1,              -1,   0,   1 },
  { TEL_VEHICLE, TEL_STEER,    -1,              -1,   0,   1 },
  { TEL_VEHICLE, TEL_LIGHTS,   -1,              -1,   0,   1 },
  { TEL_VEHICLE, TEL_KNOB_L,   -1,              -1,   0,   1 },
  { TEL_VEHICLE, TEL_KNOB_R,   -1,              -1,   0,   1 },
  { TEL_PWM,     TEL_STEER,    PIN_STEER_SERVO, CH_STEER_SERVO, 204, 409 },
};
```

The header file defines the row and the readers. The `.ino` only fills the table and answers the built-in signals.

```
enum TelKind : uint8_t {
  TEL_VEHICLE = 0,
  TEL_ADC,
  TEL_GPIO,
  TEL_PWM,
};

enum TelVehicle : uint8_t {
  TEL_TEMP = 0,
  TEL_THROTTLE,
  TEL_BATT,
  TEL_STEER,
  TEL_LIGHTS,
  TEL_KNOB_L,
  TEL_KNOB_R,
  TEL_RY,
};

struct TelChannel {
  TelKind kind;
  TelVehicle vehicle;
  int pin;
  int ledcChannel;
  int dutyMin;
  int dutyMax;
};
```

Kind	Fields that matter	Byte that is sent
`TEL_VEHICLE`	`vehicle`	`telemetryVehicle()` in the `.ino`. `pin` is ignored.
`TEL_ADC`	`pin`	`analogRead`, 0–4095 scaled to 0–255. Use a GPIO that can be an input, such as 34 or 35.
`TEL_GPIO`	`pin`	`digitalRead`: 0 or 255. An output pin reports the level last written.
`TEL_PWM`	`pin`, `ledcChannel`, `dutyMin`, `dutyMax`	LEDC duty already running on that pin. `dutyMin` becomes 0 and `dutyMax` becomes 255.

Stock rows, top to bottom:

Phone	Wire	Stock source
CH1	v0	Demo temperature, constant 90
CH2	v1	Throttle magnitude |g_ly|. Direction is not in the byte.
CH3	v2	Demo battery 76%, scaled to 0–255
CH4	v3	Steering magnitude |g_rx|
CH5	v4	Lights, switch bit 0, as 0 or 255
CH6	v5	Left knob, 0–1023 scaled to 0–255
CH7	v6	Right knob, same scale
CH8	v7	Steering-servo PWM. Duty 204. . . 409 is the 1000–2000 µs travel. Center is about 128.

```
uint8_t telemetryVehicle(TelVehicle id) {
  switch (id) {
    case TEL_TEMP:
      return (uint8_t)DEMO_MOTOR_TEMP;
    case TEL_THROTTLE:
      return (uint8_t)constrain(map(abs(g_ly), 0, 100, 0, 255), 0, 255);
    case TEL_BATT:
      return (uint8_t)constrain(map(DEMO_BATT_PCT, 0, 100, 0, 255), 0, 255);
    case TEL_STEER:
      return (uint8_t)constrain(map(abs(g_rx), 0, 100, 0, 255), 0, 255);
    case TEL_LIGHTS:
      return (g_sw & 0x01) ? 255 : 0;
    case TEL_KNOB_L:
      return (uint8_t)map(constrain(g_lk, 0, 1023), 0, 1023, 0, 255);
    case TEL_KNOB_R:
      return (uint8_t)map(constrain(g_rk, 0, 1023), 0, 1023, 0, 255);
    case TEL_RY:
      return (uint8_t)constrain(map(abs(g_ry), 0, 100, 0, 255), 0, 255);
    default:
      return 0;
  }
}
```

`TEL_RY` is implemented and unused by the stock table. To show the right-stick Y axis, replace a row's `vehicle` with `TEL_RY`. Throttle, steer, and `TEL_RY` are magnitudes: full left and full right are the same byte. A signed axis needs a center of 128, for example `map(g_ly, -100, 100, 0, 255)`, written in `telemetryVehicle()` for that one signal.

A real battery on GPIO 34 replaces the CH3 row. The vehicle name on an ADC row is ignored:

```
{ TEL_ADC, TEL_BATT, 34, -1, 0, 1 },
```

An 8-bit motor PWM on GPIO 25, LEDC channel 0, is:

```
{ TEL_PWM, TEL_THROTTLE, 25, 0, 0, 255 },
```

The pin must already be attached with `ledcAttach` (Arduino-ESP32 3.x) or `ledcSetup` plus `ledcAttachPin` (2.x) before the plot reads it. The reader itself is in the header and is the same for every row.

```
inline uint8_t telemetryReadPwm(const TelChannel& ch) {
  if (ch.dutyMax <= ch.dutyMin) return 0;
  uint32_t duty = 0;
#if ESP_ARDUINO_VERSION_MAJOR >= 3
  if (ch.pin >= 0) duty = ledcRead((uint8_t)ch.pin);
#else
```

```
  if (ch.ledcChannel >= 0) duty = ledcRead((uint8_t)ch.ledcChannel);
#endif
  long scaled = constrain((long)duty, ch.dutyMin, ch.dutyMax);
  return (uint8_t)map(scaled, ch.dutyMin, ch.dutyMax, 0, 255);
}

inline uint8_t telemetryReadOne(const TelChannel& ch, TelVehicleFn vehicle) {
  switch (ch.kind) {
    case TEL_ADC:
      if (ch.pin < 0) return 0;
      return (uint8_t)map(constrain(analogRead(ch.pin), 0, 4095), 0, 4095, 0, 255);
    case TEL_GPIO:
      if (ch.pin < 0) return 0;
      return digitalRead(ch.pin) ? 255 : 0;
    case TEL_PWM:
      return telemetryReadPwm(ch);
    default:
      return vehicle ? vehicle(ch.vehicle) : 0;
  }
}

inline void telemetryReadAll(uint8_t out[8], const TelChannel table[8], TelVehicleFn vehicle) {
  for (int i = 0; i < 8; i++) out[i] = telemetryReadOne(table[i], vehicle);
}
```

On core 3.x, `ledcRead` takes the GPIO number. On 2.x it takes the channel number. That is why a PWM row stores both `pin` and `ledcChannel`.

Changing a row does not add packets. The plot line is still eight numbers every 50 ms, about 80 bytes, about 1.6 kB/s. The phone channel menus bind CH1–CH8. There is nowhere to put a CH9.

7 Steering servo

```
// === STEERING SERVO ===
// DevKit: set ENABLE_STEER_SERVO 1 and wire a servo on PIN_STEER_SERVO.
// rx = 0 is always 1500 µs. Phone trim already offsets rx before it is sent.
#ifndef ENABLE_STEER_SERVO
#define ENABLE_STEER_SERVO 1
#endif
#ifndef PIN_STEER_SERVO
#define PIN_STEER_SERVO 13
#endif
#ifndef CH_STEER_SERVO
#define CH_STEER_SERVO 2
#endif

// Phone CH1...CH8. Change a row to TEL_ADC, TEL_GPIO, or TEL_PWM and set the pin.
// Examples:
//   { TEL_ADC,  TEL_TEMP,  34, -1, 0, 1 },            // analog pin 34
//   { TEL_GPIO, TEL_LIGHTS, 15, -1, 0, 1 },           // digital pin 15
//   { TEL_PWM,  TEL_THROTTLE, 25, 0, 0, 255 },        // 8-bit motor PWM on GPIO 25
static const TelChannel kTelemetryChannels[8] = {
  { TEL_VEHICLE, TEL_TEMP,     -1,              -1,   0,   1 },
  { TEL_VEHICLE, TEL_THROTTLE, -1,              -1,   0,   1 },
  { TEL_VEHICLE, TEL_BATT,     -1,              -1,   0,   1 },
  { TEL_VEHICLE, TEL_STEER,    -1,              -1,   0,   1 },
  { TEL_VEHICLE, TEL_LIGHTS,   -1,              -1,   0,   1 },
  { TEL_VEHICLE, TEL_KNOB_L,   -1,              -1,   0,   1 },
  { TEL_VEHICLE, TEL_KNOB_R,   -1,              -1,   0,   1 },
  { TEL_PWM,     TEL_STEER,    PIN_STEER_SERVO, CH_STEER_SERVO, 204, 409 },
};

uint8_t telemetryVehicle(TelVehicle id) {
```

```
  switch (id) {
    case TEL_TEMP:
      return (uint8_t)DEMO_MOTOR_TEMP;
    case TEL_THROTTLE:
      return (uint8_t)constrain(map(abs(g_ly), 0, 100, 0, 255), 0, 255);
    case TEL_BATT:
      return (uint8_t)constrain(map(DEMO_BATT_PCT, 0, 100, 0, 255), 0, 255);
    case TEL_STEER:
      return (uint8_t)constrain(map(abs(g_rx), 0, 100, 0, 255), 0, 255);
    case TEL_LIGHTS:
      return (g_sw & 0x01) ? 255 : 0;
    case TEL_KNOB_L:
      return (uint8_t)map(constrain(g_lk, 0, 1023), 0, 1023, 0, 255);
    case TEL_KNOB_R:
      return (uint8_t)map(constrain(g_rk, 0, 1023), 0, 1023, 0, 255);
    case TEL_RY:
      return (uint8_t)constrain(map(abs(g_ry), 0, 100, 0, 255), 0, 255);
    default:
      return 0;
  }
}

constexpr int kSteerCenterUs = 1500;
constexpr int kSteerTravelUs = 500;

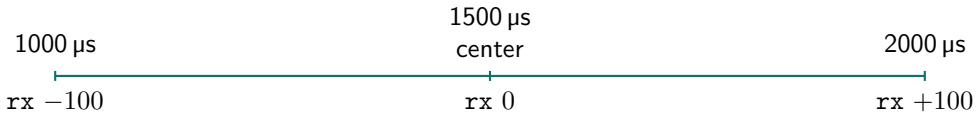
uint32_t steerDutyFromUs(int us) {
  int pulse = constrain(us, 500, 2500);
  return (uint32_t)((long)pulse * 4095L / 20000L);
}

uint32_t steerDutyFromRx(int rx) {
  int s = constrain(rx, -100, 100);
  int us = (int)map(s, -100, 100, kSteerCenterUs - kSteerTravelUs, kSteerCenterUs + kSteerTravelUs);
  return steerDutyFromUs(us);
}

#if ENABLE_STEER_SERVO
void steerServoBegin() {
#if ESP_ARDUINO_VERSION_MAJOR >= 3
  ledcAttach(PIN_STEER_SERVO, 50, 12);
  ledcWrite(PIN_STEER_SERVO, steerDutyFromUs(kSteerCenterUs));
#else
  ledcSetup(CH_STEER_SERVO, 50, 12);
  ledcAttachPin(PIN_STEER_SERVO, CH_STEER_SERVO);
  ledcWrite(CH_STEER_SERVO, steerDutyFromUs(kSteerCenterUs));
#endif
}

void steerServoWriteRx(int rx) {
  uint32_t duty = steerDutyFromRx(rx);
#if ESP_ARDUINO_VERSION_MAJOR >= 3
  ledcWrite(PIN_STEER_SERVO, duty);
#else
  ledcWrite(CH_STEER_SERVO, duty);
#endif
}
#else
void steerServoBegin() {}
void steerServoWriteRx(int rx) { (void)rx; }
#endif
```

The servo is on by default, GPIO 13, LEDC channel 2, 50 Hz, 12-bit duty. `rx = 0` is 1500 µs. Full stick is ±500 µs, so the pulse runs from 1000 µs to 2000 µs. The duty is pulse × 4095/20000, because a 20 ms frame at 12 bits is 4096 steps.



Phone trim already moves `rx` before this function sees it. `steerServoWriteRx(0)` in the fail-safe is center, not "whatever trim was last".

To move the servo to another pin, change `PIN_STEER_SERVO`. Keep `CH_STEER_SERVO` off the channels you later use for motor PWM (the binary sketches use 0 and 1 for the two drive motors, 2 for steer, 3 for pan). To build a board with no servo, set `ENABLE_STEER_SERVO` to 0. The two functions become empty, and CH8 will read 0 because no PWM is attached.

Core 3.x writes the LEDC channel by pin number. Core 2.x writes it by channel number. Both branches are in `steerServoBegin()` and `steerServoWriteRx()`.

8 Your actuators

```
178 void onControl(int lx, int ly, int rx, int ry, int lk, int rk, uint8_t sw) {
179   debugPrintControl(lx, ly, rx, ry, lk, rk, sw);
180
181   // rx = 0 → 1500 µs. Trim ticks on the phone already shift this value.
182   steerServoWriteRx(rx);
183
184   // YOUR CODE HERE - drive motors / other actuators from sticks & knobs
185   //   ly = throttle (-100..100), rx = steering, rk = pan, sw bit0 = lights
186   (void)lx; (void)ly; (void)ry; (void)lk; (void)rk; (void)sw;
187 }
188
189 void onButton(int id) {
190   debugPrintButton(id);
191
192   // YOUR CODE HERE - button actions
193   if (id == 2) {
194     // Horn note - pulse buzzer briefly
195   } else if (id == 4) {
196     // Center pan note - move camera pan servo to center
197   }
198 }
```

`onControl()` runs once per `RC:CTRL` line, which the phone sends many times a second while a stick is held. Put continuous outputs here: motor PWM, a pan servo from `rk`, a lights pin from `sw & 0x01`. The steering call above the comment is already done. The `(void)` casts only silence unused-parameter warnings; delete a cast when you start using that value.

`onButton()` runs once per press. The phone button id is the argument. The stock comments reserve id 2 for a short horn pulse and id 4 for centering a pan servo. Those bodies are empty on purpose. A button that must stay down for the whole hold is a better fit as a switch bit in `onControl()`, because `RC:BTN` is an edge, not a level.

```
409 void applyFailSafe() {
410   g_lx = g_ly = g_rx = g_ry = 0;
411   g_lk = g_rk = 512;
412   g_sw = 0;
413   steerServoWriteRx(0);
414   DBG_PRINTLN("[IO] fail-safe");
415   // YOUR CODE HERE - stop motors on disconnect
416 }
```

On disconnect, sticks, knobs, and switches return to the same defaults as boot, and the servo is written to center. Motor PWM you started in `onControl()` is *not* cleared unless you stop it under

the second `YOUR CODE HERE`. A driver left at the last throttle will keep running after the phone is gone.

9 setup

```
200 void setup() {
201   Serial.begin(SERIAL_BAUD);
202   delay(200);
203
204   steerServoBegin();
205
206   // Legacy PIN pairing only (Android often fails if SSP + setPin are mixed).
207   SerialBT.disableSSP();
208   SerialBT.setPin(BT_PIN, 4);
209
210   // isMaster=false, disableBLE=true → Classic SPP only
211   if (!SerialBT.begin(BT_NAME, false, true)) {
212     Serial.println("Bluetooth init failed!");
213     while (true) { delay(1000); }
214   }
215
216   DBG_PRINTLN();
217   DBG_PRINTLN("=== TeleCon RC Vehicle Pro - Classic Simple ===");
218   DBG_PRINT("BT name: ");
219   DBG_PRINTLN(BT_NAME);
220   DBG_PRINTLN("Pair with PIN 1234 if the phone asks.");
221   DBG_PRINTLN("Android: RC Vehicle Pro → Board DevKit → Normal → Classic + Simple");
222   DBG_PRINTLN("Handshake: RC:CONNECT,proto,simple  →  RC:ACK,app,RC");
223   DBG_PRINTLN("Examples:");
224   DBG_PRINTLN("  RC:CONNECT,proto,simple");
225   DBG_PRINTLN("  RC:CTRL,lx,0,ly,0,rx,0,ry,0,lk,512,rk,512,sw,00");
226   DBG_PRINTLN("  RC:BTN,id,1");
227   DBG_PRINTLN("Steer: rx=0 is 1500 µs. Trim stays on the phone.");
228   DBG_PRINTLN("Edit onControl() / onButton() at YOUR CODE HERE.");
229   DBG_PRINTLN("Telemetry: RC:DATA every 500 ms (speed×10 from |ly|, demo batt/temp).");
230 #if ENABLE_PLOT
231   DBG_PRINTLN("Plot ON - RC:PLOT ~50 ms, CH1...CH8 (edit kTelemetryChannels[]).");
232 #else
233   DBG_PRINTLN("Plot OFF - set ENABLE_PLOT 1 to send RC:PLOT.");
234 #endif
235 #if ENABLE_SERIAL_INJECT
236   DBG_PRINTLN("Serial inject @ 115200 (after handshake, line ending = Newline):");
237   DBG_PRINTLN("  RC:DATA,left,186,right,0,lo,1,ro,0,lg,1,rg,0,analog,90,batt,76,led,01");
238   DBG_PRINTLN("  RC:PLOT,v0,128,v1,200,v2,64,v3,180,v4,0,v5,0,v6,0,v7,0");
239   DBG_PRINTLN("  Inject is one-shot (pulse); auto telemetry resumes after.");
240 #endif
241   DBG_PRINTLN();
242 }
```

Order matters. Serial starts first so a Bluetooth failure can be printed. The servo is centered before the radio comes up, so a link that never connects still leaves the wheel straight. `disableSSP()` plus `setPin(..., 4)` is legacy PIN pairing. Mixing that with Secure Simple Pairing is a known way to make Android fail the bond, which is why SSP is turned off. `SerialBT.begin(name, false, true)` means "not the master" and "Classic only, BLE off". If `begin` fails, the sketch stops in `while (true)`.

The banner is entirely debug. It does not change the protocol. It prints the lines you can paste into the Serial Monitor when inject is on.

10 loop

```
void loop() {
  bool connected = SerialBT.hasClient();
  if (connected != btWasConnected) {
    btWasConnected = connected;
    if (connected) {
      resetHandshake();
      DBG_PRINTLN("[BT] connected - waiting for RC:CONNECT,proto,simple");
    } else {
      applyFailSafe();
      resetHandshake();
      DBG_PRINTLN("[BT] disconnected");
    }
  }

  while (SerialBT.available()) {
    String line = SerialBT.readStringUntil('\n');
    line.trim();
    if (line.length() > 0) {
      handleLine(line);
    }
  }

#if ENABLE_SERIAL_INJECT
  serialInjectPoll();
#endif

  if (!handshakeOk) return;

  unsigned long now = millis();

  if (now - lastDataMs >= TELEMETRY_DATA_MS) {
    lastDataMs = now;

    int speedX10 = abs(g_ly) * 25 / 10;
    int analog = DEMO_MOTOR_TEMP;
    int batt = DEMO_BATT_PCT;

    char data[128];
    snprintf(data, sizeof(data),
      "RC:DATA,left,%d,right,0,lo,1,ro,0,lg,1,rg,0,analog,%d,batt,%d,led,%02X",
      speedX10, analog, batt, g_sw);
    sendLine(data);
  }

#if ENABLE_PLOT
  if (now - lastPlotMs >= PLOT_MS) {
    lastPlotMs = now;

    uint8_t sample[8];
    telemetryReadAll(sample, kTelemetryChannels, telemetryVehicle);
    char plot[128];
    snprintf(plot, sizeof(plot),
      "RC:PLOT,v0,%u,v1,%u,v2,%u,v3,%u,v4,%u,v5,%u,v6,%u,v7,%u",
      sample[0], sample[1], sample[2], sample[3],
      sample[4], sample[5], sample[6], sample[7]);
    sendLine(plot);
  }
#endif
}
```

Each pass does four things, in this order.

1. Watch the Classic client. A new client clears the handshake so an old session cannot stay

"ready" across a reconnect. A drop calls `applyFailSafe()`.

2. Read every complete line from Bluetooth and hand it to `handleLine()`. The delimiter is `\n`. A line with no characters after `trim` is ignored.

3. If inject is compiled in, take one newline-terminated line from the Serial Monitor.

4. If the handshake is still open, return. No `RC:DATA` and no `RC:PLOT` until `RC:ACK` has been sent.

After the gate, two timers run from `millis()`. They do not reset each other.

`RC:DATA` every 500 ms is the vehicle HUD, not the eight plot channels. `left` is $|\texttt{g_ly}| \times 2.5$, sent as an integer (`abs(g_ly) * 25 / 10`): full throttle 100 becomes 250, which the phone treats as speed $\times 10$. `right` is always 0 in this starter. `lo,1,ro,0,lg,1,rg,0` are fixed panel lamps. `analog` is the demo temperature, `batt` the demo percentage, `led` the switch byte in two hex digits. Replace `analog` and `batt` in this `snprintf` when the HUD should follow the same sensor as CH1 or CH3. The plot table does not fill this line by itself.

`RC:PLOT` every 50 ms calls `telemetryReadAll`, which walks the eight rows. The line is always `v0` through `v7`, in that order.

11 Handshake and the text parser

```
String getValue(const String& line, const char* key) {
  int colon = line.indexOf(':');
  if (colon < 0) return "";

  int pos = line.indexOf(',', colon);
  if (pos < 0) return "";
  pos++;

  while (pos < (int)line.length()) {
    int keyEnd = line.indexOf(',', pos);
    if (keyEnd < 0) break;

    String k = line.substring(pos, keyEnd);
    int valEnd = line.indexOf(',', keyEnd + 1);
    String v = (valEnd < 0)
      ? line.substring(keyEnd + 1)
      : line.substring(keyEnd + 1, valEnd);

    if (k == key) return v;
    pos = (valEnd < 0) ? line.length() : valEnd + 1;
  }
  return "";
}

void handleConnect(const String& app, const String& line) {
  if (!app.equalsIgnoreCase(APP_PREFIX)) {
    sendLine(String("RC:NAK,reason,app_mismatch,expected,") + APP_PREFIX +
             ",actual," + app);
    handshakeOk = false;
    return;
  }

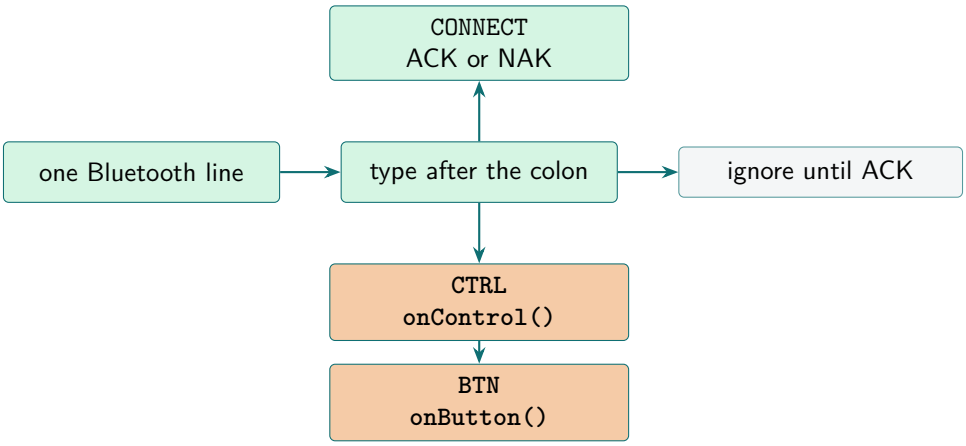
  String proto = getValue(line, "proto");
  if (proto.length() == 0) {
    sendLine(String("RC:NAK,reason,proto_mismatch,expected,") + PROTO_WIRE +
             ",actual,unknown");
    handshakeOk = false;
    return;
  }
```

```
464
465   if (proto.equalsIgnoreCase(PROTO_WIRE)) {
466     sendLine("RC:ACK,app,RC");
467     handshakeOk = true;
468     DBG_PRINTLN("[HS] ACK - session ready");
469     return;
470   }
471
472   sendLine(String("RC:NAK,reason,proto_mismatch,expected,") + PROTO_WIRE +
473            ",actual," + proto);
474   handshakeOk = false;
475 }
476
477 void handleLine(const String& line) {
478   int colon = line.indexOf(':');
479   if (colon <= 0) return;
480
481   String app = line.substring(0, colon);
482   String rest = line.substring(colon + 1);
483   int comma = rest.indexOf(',');
484   String type = (comma < 0) ? rest : rest.substring(0, comma);
485
486   if (type.equalsIgnoreCase("CONNECT")) {
487     handleConnect(app, line);
488     return;
489   }
490
491   if (!handshakeOk) return;
492   if (!app.equalsIgnoreCase("RC")) return;
493
494   if (type.equalsIgnoreCase("CTRL")) {
495     g_lx = getValue(line, "lx").toInt();
496     g_ly = getValue(line, "ly").toInt();
497     g_rx = getValue(line, "rx").toInt();
498     g_ry = getValue(line, "ry").toInt();
499     g_lk = getValue(line, "lk").toInt();
500     g_rk = getValue(line, "rk").toInt();
501     String swHex = getValue(line, "sw");
502     g_sw = (uint8_t)strtol(swHex.c_str(), nullptr, 16);
503
504     onControl(g_lx, g_ly, g_rx, g_ry, g_lk, g_rk, g_sw);
505   } else if (type.equalsIgnoreCase("BTN")) {
506     onButton(getValue(line, "id").toInt());
507   }
508 }
```

`getValue()` understands one shape: `RC:TYPE,key,value,key,value`. It skips the `APP:TYPE` head, then walks pairs. The value runs to the next comma, or to the end of the line for the last pair. Keys are case sensitive (`lx`, not `LX`). A missing key returns an empty string, and `toInt()` of that string is 0, so a broken `RC:CTRL` looks like centered sticks.

`handleConnect()` checks two things. The app name before the colon must be `RC`. The `proto` value must be `simple`. Anything else is `RC:NAK` with `app_mismatch` or `proto_mismatch`, and `handshakeOk` stays false. Success is exactly `RC:ACK,app,RC`.

`handleLine()` ignores a line with no colon. `CONNECT` is accepted before the session is ready; every other type is dropped until then. After the ACK, only an `RC` app is accepted. `CTRL` stores the seven fields and calls `onControl()`. `BTN` reads `id` and calls `onButton()`. Any other type (`DATA`, `PLOT`, `SET`) is ignored on the way in. This sketch sends `DATA` and `PLOT`; it does not obey them.



12 Serial inject and the monitor

```
#if ENABLE_SERIAL_INJECT
void handleSerialInjectLine(const String& line) {
  if (!SerialBT.hasClient()) {
    DBG_PRINTLN("[SERIAL] no BT client - line not sent");
    return;
  }
  if (!handshakeOk) {
    DBG_PRINTLN("[SERIAL] waiting for RC:CONNECT handshake before inject");
    return;
  }

  if (line.startsWith("RC:DATA") || line.startsWith("RC:PLOT")) {
    sendLine(line);
    DBG_PRINT("[SERIAL] inject one-shot → phone: ");
    DBG_PRINTLN(line.startsWith("RC:DATA") ? "RC:DATA" : "RC:PLOT");
    return;
  }

  DBG_PRINT("[SERIAL] unhandled inject (use RC:DATA / RC:PLOT): ");
  DBG_PRINTLN(line);
}

void serialInjectPoll() {
  while (Serial.available()) {
    char c = (char)Serial.read();
    if (c == '\n') {
      if (serialLineBuf.length() > 0) {
        serialLineBuf.trim();
        if (serialLineBuf.length() > 0) {
          handleSerialInjectLine(serialLineBuf);
        }
        serialLineBuf = "";
      }
    } else if (c != '\r') {
      if ((int)serialLineBuf.length() < SERIAL_MAX_LINE) {
        serialLineBuf += c;
      } else {
        serialLineBuf = "";
        DBG_PRINTLN("[SERIAL] line too long, discarded");
      }
    }
  }
}
#endif
```

Inject is a bench tool. It does not run when Bluetooth has no client, and it does not run before the handshake. Only a line that starts with `RC:DATA` or `RC:PLOT` is written to the phone, once. The 500 ms and 50 ms timers are not stopped, so the following automatic line overwrites that inject on the phone. A line longer than 192 characters is thrown away. `\r` is stripped so a monitor set to "Both NL & CR" still works; the sketch commits on `\n`.

```
uint16_t stickTo12(int s) {
  return (uint16_t)(((constrain(s, -100, 100) + 100) * 4095) / 200);
}

void debugPrintControl(int lx, int ly, int rx, int ry, int lk, int rk, uint8_t sw) {
  uint16_t lx12 = stickTo12(lx);
  uint16_t ly12 = stickTo12(ly);
  uint16_t rx12 = stickTo12(rx);
  uint16_t ry12 = stickTo12(ry);
  uint16_t lkU = (uint16_t)constrain(lk, 0, 1023);
  uint16_t rkU = (uint16_t)constrain(rk, 0, 1023);

  if (lx12 != lastPrintLX) {
    DBG_PRINTF("L Stick X: %u    ", lx12);
    DBG_PRINTF("L Stick Y: %u\n", ly12);
    lastPrintLX = lx12;
  }
  if (ly12 != lastPrintLY) {
    DBG_PRINTF("L Stick X: %u    ", lx12);
    DBG_PRINTF("L Stick Y: %u\n", ly12);
    lastPrintLY = ly12;
  }
  if (rx12 != lastPrintRX) {
    DBG_PRINTF("R Stick X: %u    ", rx12);
    DBG_PRINTF("R Stick Y: %u\n", ry12);
    lastPrintRX = rx12;
  }
  if (ry12 != lastPrintRY) {
    DBG_PRINTF("R Stick X: %u    ", rx12);
    DBG_PRINTF("R Stick Y: %u\n", ry12);
    lastPrintRY = ry12;
  }
  if (lkU != lastPrintKnobL) {
    DBG_PRINTF("Left Knob: %u\n", lkU);
    lastPrintKnobL = lkU;
  }
  if (rkU != lastPrintKnobR) {
    DBG_PRINTF("Right Knob: %u\n", rkU);
    lastPrintKnobR = rkU;
  }
  if (sw != lastPrintSw) {
    for (int i = 0; i < 6; i++) {
      byte bit = (1 << i);
      if ((sw & bit) != (lastPrintSw & bit)) {
        DBG_PRINTF("SW%d = %d\n", i + 1, (sw & bit) ? 1 : 0);
      }
    }
    lastPrintSw = sw;
  }
}

void debugPrintButton(int id) {
  DBG_PRINTF("BTN%d : Pressed\n", id);
}
```

`stickTo12()` maps $-100 \ldots 100$ onto $0 \ldots 4095$ for the monitor only. 0 on the stick is about 2048 on this scale. That number is not the servo duty and not the plot byte. `debugPrintControl()`

prints a stick when that axis changes. It prints SW1–SW6 when a bit flips. debugPrintButton() prints the id. With ENABLE_DEBUG at 0 these functions still run, but the macros produce no output.

sendLine() is the only writer to the phone. It prints the text and then a newline.

resetHandshake() does not move the servo. The caller that detected the drop already called applyFailSafe().

```
404 void sendLine(const String& line) {
405   SerialBT.print(line);
406   SerialBT.print('\n');
407 }
408
409 void applyFailSafe() {
410   g_lx = g_ly = g_rx = g_ry = 0;
411   g_lk = g_rk = 512;
412   g_sw = 0;
413   steerServoWriteRx(0);
414   DBG_PRINTLN("[IO] fail-safe");
415   // YOUR CODE HERE - stop motors on disconnect
416 }
417
418 void resetHandshake() {
419   handshakeOk = false;
420   lastPrintLX = lastPrintLY = lastPrintRX = lastPrintRY = 0xFFFF;
421   lastPrintKnobL = lastPrintKnobR = 0xFFFF;
422   lastPrintSw = 0xFF;
423 }
```

13 A short checklist before you flash

1. Board is an ESP32 Dev Module, not an ESP32-CAM.
2. Phone connection is Classic + Simple, name ESP32-TC-RC-BT-Simple.
3. Motor code is in onControl(), and the same outputs are stopped in applyFailSafe().
4. kTelemetryChannels[] still has eight rows.
5. A TEL_PWM row uses a pin that setup (or your code) has already attached.
6. Servo signal is GPIO 13, or you changed PIN_STEER_SERVO and the wire together.
7. After the link works, set the three ENABLE_* switches you no longer need, and upload again.

A Full TeleCon_Classic_Simple.ino

The listing is the file Arduino compiles. Line numbers match the sections above.

```
1  /*
2   * TeleCon RC Vehicle Pro - Classic Simple (ESP32 Dev Module, no camera)
3   *
4   * Android (TeleCon4ESP32):
5   *   1. Pair Bluetooth "ESP32-TC-RC-BT-Simple" (PIN 1234 if asked)
6   *   2. Control Panel or RC Vehicle Pro → Board DevKit → Classic + Simple
7   *   3. Connect - phone sends RC:CONNECT,proto,simple
8   *
9   * Single-file starter for Normal users. Edit onControl() / onButton().
10  * Advanced Classic Binary: ../../Classic_Binary/
11  */
12
13  #include "BluetoothSerial.h"
```

```
14  #include "TelemetryChannels.h"
15
16  // === CONFIG ===
17  #define BT_NAME             "ESP32-TC-RC-BT-Simple"
18  #define BT_PIN              "1234"
19  #define APP_PREFIX          "RC"
20  #define PROTO_WIRE          "simple"
21  #define SERIAL_BAUD         115200
22
23  // 1 = Serial banners + stick/HS/IO logs. Set 0 when Connect works.
24  #define ENABLE_DEBUG        1
25
26  #if ENABLE_DEBUG
27    #define DBG_PRINT(x)      Serial.print(x)
28    #define DBG_PRINTLN(x)    Serial.println(x)
29    #define DBG_PRINTF(...)   Serial.printf(__VA_ARGS__)
30  #else
31    #define DBG_PRINT(x)      ((void)0)
32    #define DBG_PRINTLN(x)    ((void)0)
33    #define DBG_PRINTF(...)   ((void)0)
34  #endif
35
36  #define TELEMETRY_DATA_MS   500
37  #define ENABLE_PLOT         1
38  #define PLOT_MS             50
39
40  #define ENABLE_SERIAL_INJECT 1
41  #define SERIAL_MAX_LINE     192
42
43  // Demo HUD values (replace with ADC reads when you wire sensors)
44  #define DEMO_MOTOR_TEMP     90
45  #define DEMO_BATT_PCT       76
46
47  // === BLUETOOTH ===
48  BluetoothSerial SerialBT;
49
50  bool handshakeOk = false;
51  bool btWasConnected = false;
52
53  int g_lx = 0, g_ly = 0, g_rx = 0, g_ry = 0;
54  int g_lk = 512, g_rk = 512;
55  uint8_t g_sw = 0;
56
57  unsigned long lastDataMs = 0;
58  unsigned long lastPlotMs = 0;
59
60  #if ENABLE_SERIAL_INJECT
61  static String serialLineBuf;
62  #endif
63
64  static uint16_t lastPrintLX = 0xFFFF;
65  static uint16_t lastPrintLY = 0xFFFF;
66  static uint16_t lastPrintRX = 0xFFFF;
67  static uint16_t lastPrintRY = 0xFFFF;
68  static uint16_t lastPrintKnobL = 0xFFFF;
69  static uint16_t lastPrintKnobR = 0xFFFF;
70  static uint8_t lastPrintSw = 0xFF;
71
72  String getValue(const String& line, const char* key);
73  void handleLine(const String& line);
74  void handleConnect(const String& app, const String& line);
75  void sendLine(const String& line);
76  void resetHandshake();
77  void applyFailSafe();
78  void debugPrintControl(int lx, int ly, int rx, int ry, int lk, int rk, uint8_t sw);
79  void debugPrintButton(int id);
80  uint16_t stickTo12(int s);
81  #if ENABLE_SERIAL_INJECT
82  void serialInjectPoll();
83  void handleSerialInjectLine(const String& line);
84  #endif
85
86
87  // === STEERING SERVO ===
88  // DevKit: set ENABLE_STEER_SERVO 1 and wire a servo on PIN_STEER_SERVO.
```

```
// rx = 0 is always 1500 µs. Phone trim already offsets rx before it is sent.
#ifndef ENABLE_STEER_SERVO
#define ENABLE_STEER_SERVO 1
#endif
#ifndef PIN_STEER_SERVO
#define PIN_STEER_SERVO 13
#endif
#ifndef CH_STEER_SERVO
#define CH_STEER_SERVO 2
#endif

// Phone CH1...CH8. Change a row to TEL_ADC, TEL_GPIO, or TEL_PWM and set the pin.
// Examples:
//   { TEL_ADC,  TEL_TEMP,  34, -1, 0, 1 },            // analog pin 34
//   { TEL_GPIO, TEL_LIGHTS, 15, -1, 0, 1 },           // digital pin 15
//   { TEL_PWM,  TEL_THROTTLE, 25, 0, 0, 255 },        // 8-bit motor PWM on GPIO 25
static const TelChannel kTelemetryChannels[8] = {
  { TEL_VEHICLE, TEL_TEMP,     -1,              -1,   0,   1 },
  { TEL_VEHICLE, TEL_THROTTLE, -1,              -1,   0,   1 },
  { TEL_VEHICLE, TEL_BATT,     -1,              -1,   0,   1 },
  { TEL_VEHICLE, TEL_STEER,    -1,              -1,   0,   1 },
  { TEL_VEHICLE, TEL_LIGHTS,   -1,              -1,   0,   1 },
  { TEL_VEHICLE, TEL_KNOB_L,   -1,              -1,   0,   1 },
  { TEL_VEHICLE, TEL_KNOB_R,   -1,              -1,   0,   1 },
  { TEL_PWM,     TEL_STEER,    PIN_STEER_SERVO, CH_STEER_SERVO, 204, 409 },
};

uint8_t telemetryVehicle(TelVehicle id) {
  switch (id) {
    case TEL_TEMP:
      return (uint8_t)DEMO_MOTOR_TEMP;
    case TEL_THROTTLE:
      return (uint8_t)constrain(map(abs(g_ly), 0, 100, 0, 255), 0, 255);
    case TEL_BATT:
      return (uint8_t)constrain(map(DEMO_BATT_PCT, 0, 100, 0, 255), 0, 255);
    case TEL_STEER:
      return (uint8_t)constrain(map(abs(g_rx), 0, 100, 0, 255), 0, 255);
    case TEL_LIGHTS:
      return (g_sw & 0x01) ? 255 : 0;
    case TEL_KNOB_L:
      return (uint8_t)map(constrain(g_lk, 0, 1023), 0, 1023, 0, 255);
    case TEL_KNOB_R:
      return (uint8_t)map(constrain(g_rk, 0, 1023), 0, 1023, 0, 255);
    case TEL_RY:
      return (uint8_t)constrain(map(abs(g_ry), 0, 100, 0, 255), 0, 255);
    default:
      return 0;
  }
}

constexpr int kSteerCenterUs = 1500;
constexpr int kSteerTravelUs = 500;

uint32_t steerDutyFromUs(int us) {
  int pulse = constrain(us, 500, 2500);
  return (uint32_t)((long)pulse * 4095L / 20000L);
}

uint32_t steerDutyFromRx(int rx) {
  int s = constrain(rx, -100, 100);
  int us = (int)map(s, -100, 100, kSteerCenterUs - kSteerTravelUs, kSteerCenterUs + kSteerTravelUs);
  return steerDutyFromUs(us);
}

#if ENABLE_STEER_SERVO
void steerServoBegin() {
#if ESP_ARDUINO_VERSION_MAJOR >= 3
  ledcAttach(PIN_STEER_SERVO, 50, 12);
  ledcWrite(PIN_STEER_SERVO, steerDutyFromUs(kSteerCenterUs));
#else
  ledcSetup(CH_STEER_SERVO, 50, 12);
  ledcAttachPin(PIN_STEER_SERVO, CH_STEER_SERVO);
  ledcWrite(CH_STEER_SERVO, steerDutyFromUs(kSteerCenterUs));
#endif
}
```

```
164
165 void steerServoWriteRx(int rx) {
166   uint32_t duty = steerDutyFromRx(rx);
167 #if ESP_ARDUINO_VERSION_MAJOR >= 3
168   ledcWrite(PIN_STEER_SERVO, duty);
169 #else
170   ledcWrite(CH_STEER_SERVO, duty);
171 #endif
172 }
173 #else
174 void steerServoBegin() {}
175 void steerServoWriteRx(int rx) { (void)rx; }
176 #endif
177
178 void onControl(int lx, int ly, int rx, int ry, int lk, int rk, uint8_t sw) {
179   debugPrintControl(lx, ly, rx, ry, lk, rk, sw);
180
181   // rx = 0 → 1500 µs. Trim ticks on the phone already shift this value.
182   steerServoWriteRx(rx);
183
184   // YOUR CODE HERE - drive motors / other actuators from sticks & knobs
185   //   ly = throttle (-100..100), rx = steering, rk = pan, sw bit0 = lights
186   (void)lx; (void)ly; (void)ry; (void)lk; (void)rk; (void)sw;
187 }
188
189 void onButton(int id) {
190   debugPrintButton(id);
191
192   // YOUR CODE HERE - button actions
193   if (id == 2) {
194     // Horn note - pulse buzzer briefly
195   } else if (id == 4) {
196     // Center pan note - move camera pan servo to center
197   }
198 }
199
200 void setup() {
201   Serial.begin(SERIAL_BAUD);
202   delay(200);
203
204   steerServoBegin();
205
206   // Legacy PIN pairing only (Android often fails if SSP + setPin are mixed).
207   SerialBT.disableSSP();
208   SerialBT.setPin(BT_PIN, 4);
209
210   // isMaster=false, disableBLE=true → Classic SPP only
211   if (!SerialBT.begin(BT_NAME, false, true)) {
212     Serial.println("Bluetooth init failed!");
213     while (true) { delay(1000); }
214   }
215
216   DBG_PRINTLN();
217   DBG_PRINTLN("=== TeleCon RC Vehicle Pro - Classic Simple ===");
218   DBG_PRINT("BT name: ");
219   DBG_PRINTLN(BT_NAME);
220   DBG_PRINTLN("Pair with PIN 1234 if the phone asks.");
221   DBG_PRINTLN("Android: RC Vehicle Pro → Board DevKit → Normal → Classic + Simple");
222   DBG_PRINTLN("Handshake: RC:CONNECT,proto,simple  →  RC:ACK,app,RC");
223   DBG_PRINTLN("Examples:");
224   DBG_PRINTLN("  RC:CONNECT,proto,simple");
225   DBG_PRINTLN("  RC:CTRL,lx,0,ly,0,rx,0,ry,0,lk,512,rk,512,sw,00");
226   DBG_PRINTLN("  RC:BTN,id,1");
227   DBG_PRINTLN("Steer: rx=0 is 1500 µs. Trim stays on the phone.");
228   DBG_PRINTLN("Edit onControl() / onButton() at YOUR CODE HERE.");
229   DBG_PRINTLN("Telemetry: RC:DATA every 500 ms (speed×10 from |ly|, demo batt/temp).");
230 #if ENABLE_PLOT
231   DBG_PRINTLN("Plot ON - RC:PLOT ~50 ms, CH1...CH8 (edit kTelemetryChannels[]).");
232 #else
233   DBG_PRINTLN("Plot OFF - set ENABLE_PLOT 1 to send RC:PLOT.");
234 #endif
235 #if ENABLE_SERIAL_INJECT
236   DBG_PRINTLN("Serial inject @ 115200 (after handshake, line ending = Newline):");
237   DBG_PRINTLN("  RC:DATA,left,186,right,0,lo,1,ro,0,lg,1,rg,0,analog,90,batt,76,led,01");
238   DBG_PRINTLN("  RC:PLOT,v0,128,v1,200,v2,64,v3,180,v4,0,v5,0,v6,0,v7,0");
```

```
239     DBG_PRINTLN("  Inject is one-shot (pulse); auto telemetry resumes after.");
240   #endif
241     DBG_PRINTLN();
242   }
243
244   void loop() {
245     bool connected = SerialBT.hasClient();
246     if (connected != btWasConnected) {
247       btWasConnected = connected;
248       if (connected) {
249         resetHandshake();
250         DBG_PRINTLN("[BT] connected - waiting for RC:CONNECT,proto,simple");
251       } else {
252         applyFailSafe();
253         resetHandshake();
254         DBG_PRINTLN("[BT] disconnected");
255       }
256     }
257
258     while (SerialBT.available()) {
259       String line = SerialBT.readStringUntil('\n');
260       line.trim();
261       if (line.length() > 0) {
262         handleLine(line);
263       }
264     }
265
266   #if ENABLE_SERIAL_INJECT
267     serialInjectPoll();
268   #endif
269
270     if (!handshakeOk) return;
271
272     unsigned long now = millis();
273
274     if (now - lastDataMs >= TELEMETRY_DATA_MS) {
275       lastDataMs = now;
276
277       int speedX10 = abs(g_ly) * 25 / 10;
278       int analog = DEMO_MOTOR_TEMP;
279       int batt = DEMO_BATT_PCT;
280
281       char data[128];
282       snprintf(data, sizeof(data),
283         "RC:DATA,left,%d,right,0,lo,1,ro,0,lg,1,rg,0,analog,%d,batt,%d,led,%02X",
284         speedX10, analog, batt, g_sw);
285       sendLine(data);
286     }
287
288   #if ENABLE_PLOT
289     if (now - lastPlotMs >= PLOT_MS) {
290       lastPlotMs = now;
291
292       uint8_t sample[8];
293       telemetryReadAll(sample, kTelemetryChannels, telemetryVehicle);
294       char plot[128];
295       snprintf(plot, sizeof(plot),
296         "RC:PLOT,v0,%u,v1,%u,v2,%u,v3,%u,v4,%u,v5,%u,v6,%u,v7,%u",
297         sample[0], sample[1], sample[2], sample[3],
298         sample[4], sample[5], sample[6], sample[7]);
299       sendLine(plot);
300     }
301   #endif
302   }
303
304   #if ENABLE_SERIAL_INJECT
305   void handleSerialInjectLine(const String& line) {
306     if (!SerialBT.hasClient()) {
307       DBG_PRINTLN("[SERIAL] no BT client - line not sent");
308       return;
309     }
310     if (!handshakeOk) {
311       DBG_PRINTLN("[SERIAL] waiting for RC:CONNECT handshake before inject");
312       return;
313     }
```

```
314
315     if (line.startsWith("RC:DATA") || line.startsWith("RC:PLOT")) {
316       sendLine(line);
317       DBG_PRINT("[SERIAL] inject one-shot → phone: ");
318       DBG_PRINTLN(line.startsWith("RC:DATA") ? "RC:DATA" : "RC:PLOT");
319       return;
320     }
321
322     DBG_PRINT("[SERIAL] unhandled inject (use RC:DATA / RC:PLOT): ");
323     DBG_PRINTLN(line);
324   }
325
326   void serialInjectPoll() {
327     while (Serial.available()) {
328       char c = (char)Serial.read();
329       if (c == '\n') {
330         if (serialLineBuf.length() > 0) {
331           serialLineBuf.trim();
332           if (serialLineBuf.length() > 0) {
333             handleSerialInjectLine(serialLineBuf);
334           }
335           serialLineBuf = "";
336         }
337       } else if (c != '\r') {
338         if ((int)serialLineBuf.length() < SERIAL_MAX_LINE) {
339           serialLineBuf += c;
340         } else {
341           serialLineBuf = "";
342           DBG_PRINTLN("[SERIAL] line too long, discarded");
343         }
344       }
345     }
346   }
347   #endif
348
349   uint16_t stickTo12(int s) {
350     return (uint16_t)(((constrain(s, -100, 100) + 100) * 4095) / 200);
351   }
352
353   void debugPrintControl(int lx, int ly, int rx, int ry, int lk, int rk, uint8_t sw) {
354     uint16_t lx12 = stickTo12(lx);
355     uint16_t ly12 = stickTo12(ly);
356     uint16_t rx12 = stickTo12(rx);
357     uint16_t ry12 = stickTo12(ry);
358     uint16_t lkU = (uint16_t)constrain(lk, 0, 1023);
359     uint16_t rkU = (uint16_t)constrain(rk, 0, 1023);
360
361     if (lx12 != lastPrintLX) {
362       DBG_PRINTF("L Stick X: %u    ", lx12);
363       DBG_PRINTF("L Stick Y: %u\n", ly12);
364       lastPrintLX = lx12;
365     }
366     if (ly12 != lastPrintLY) {
367       DBG_PRINTF("L Stick X: %u    ", lx12);
368       DBG_PRINTF("L Stick Y: %u\n", ly12);
369       lastPrintLY = ly12;
370     }
371     if (rx12 != lastPrintRX) {
372       DBG_PRINTF("R Stick X: %u    ", rx12);
373       DBG_PRINTF("R Stick Y: %u\n", ry12);
374       lastPrintRX = rx12;
375     }
376     if (ry12 != lastPrintRY) {
377       DBG_PRINTF("R Stick X: %u    ", rx12);
378       DBG_PRINTF("R Stick Y: %u\n", ry12);
379       lastPrintRY = ry12;
380     }
381     if (lkU != lastPrintKnobL) {
382       DBG_PRINTF("Left Knob: %u\n", lkU);
383       lastPrintKnobL = lkU;
384     }
385     if (rkU != lastPrintKnobR) {
386       DBG_PRINTF("Right Knob: %u\n", rkU);
387       lastPrintKnobR = rkU;
388     }
```

```
389     if (sw != lastPrintSw) {
390       for (int i = 0; i < 6; i++) {
391         byte bit = (1 << i);
392         if ((sw & bit) != (lastPrintSw & bit)) {
393           DBG_PRINTF("SW%d = %d\n", i + 1, (sw & bit) ? 1 : 0);
394         }
395       }
396       lastPrintSw = sw;
397     }
398   }
399
400   void debugPrintButton(int id) {
401     DBG_PRINTF("BTN%d : Pressed\n", id);
402   }
403
404   void sendLine(const String& line) {
405     SerialBT.print(line);
406     SerialBT.print('\n');
407   }
408
409   void applyFailSafe() {
410     g_lx = g_ly = g_rx = g_ry = 0;
411     g_lk = g_rk = 512;
412     g_sw = 0;
413     steerServoWriteRx(0);
414     DBG_PRINTLN("[IO] fail-safe");
415     // YOUR CODE HERE - stop motors on disconnect
416   }
417
418   void resetHandshake() {
419     handshakeOk = false;
420     lastPrintLX = lastPrintLY = lastPrintRX = lastPrintRY = 0xFFFF;
421     lastPrintKnobL = lastPrintKnobR = 0xFFFF;
422     lastPrintSw = 0xFF;
423   }
424
425   String getValue(const String& line, const char* key) {
426     int colon = line.indexOf(':');
427     if (colon < 0) return "";
428
429     int pos = line.indexOf(',', colon);
430     if (pos < 0) return "";
431     pos++;
432
433     while (pos < (int)line.length()) {
434       int keyEnd = line.indexOf(',', pos);
435       if (keyEnd < 0) break;
436
437       String k = line.substring(pos, keyEnd);
438       int valEnd = line.indexOf(',', keyEnd + 1);
439       String v = (valEnd < 0)
440         ? line.substring(keyEnd + 1)
441         : line.substring(keyEnd + 1, valEnd);
442
443       if (k == key) return v;
444       pos = (valEnd < 0) ? line.length() : valEnd + 1;
445     }
446     return "";
447   }
448
449   void handleConnect(const String& app, const String& line) {
450     if (!app.equalsIgnoreCase(APP_PREFIX)) {
451       sendLine(String("RC:NAK,reason,app_mismatch,expected,") + APP_PREFIX +
452                ",actual," + app);
453       handshakeOk = false;
454       return;
455     }
456
457     String proto = getValue(line, "proto");
458     if (proto.length() == 0) {
459       sendLine(String("RC:NAK,reason,proto_mismatch,expected,") + PROTO_WIRE +
460                ",actual,unknown");
461       handshakeOk = false;
462       return;
463     }
```

```
464
465     if (proto.equalsIgnoreCase(PROTO_WIRE)) {
466       sendLine("RC:ACK,app,RC");
467       handshakeOk = true;
468       DBG_PRINTLN("[HS] ACK - session ready");
469       return;
470     }
471
472     sendLine(String("RC:NAK,reason,proto_mismatch,expected,") + PROTO_WIRE +
473              ",actual," + proto);
474     handshakeOk = false;
475   }
476
477   void handleLine(const String& line) {
478     int colon = line.indexOf(':');
479     if (colon <= 0) return;
480
481     String app = line.substring(0, colon);
482     String rest = line.substring(colon + 1);
483     int comma = rest.indexOf(',');
484     String type = (comma < 0) ? rest : rest.substring(0, comma);
485
486     if (type.equalsIgnoreCase("CONNECT")) {
487       handleConnect(app, line);
488       return;
489     }
490
491     if (!handshakeOk) return;
492     if (!app.equalsIgnoreCase("RC")) return;
493
494     if (type.equalsIgnoreCase("CTRL")) {
495       g_lx = getValue(line, "lx").toInt();
496       g_ly = getValue(line, "ly").toInt();
497       g_rx = getValue(line, "rx").toInt();
498       g_ry = getValue(line, "ry").toInt();
499       g_lk = getValue(line, "lk").toInt();
500       g_rk = getValue(line, "rk").toInt();
501       String swHex = getValue(line, "sw");
502       g_sw = (uint8_t)strtol(swHex.c_str(), nullptr, 16);
503
504       onControl(g_lx, g_ly, g_rx, g_ry, g_lk, g_rk, g_sw);
505     } else if (type.equalsIgnoreCase("BTN")) {
506       onButton(getValue(line, "id").toInt());
507     }
508   }
```

B Full `TelemetryChannels.h`

```
1   #pragma once
2
3   #include <Arduino.h>
4
5   /*
6    * Phone CH1...CH8 are RC:PLOT v0...v7.
7    * Edit kTelemetryChannels[] in the sketch to choose what each channel reports (0-255).
8    *
9    * TEL_VEHICLE  built-in signal (temp, throttle, battery, steer, lights, knobs)
10   * TEL_ADC      analogRead(pin), 0-4095 scaled to 0-255
11   * TEL_GPIO     digitalRead(pin) → 0 or 255 (an output pin reports what was written)
12   * TEL_PWM      LEDC duty on pin. dutyMin maps to 0, dutyMax maps to 255.
13   *              8-bit motor PWM uses 0...255. 12-bit servo travel 1000-2000 µs uses 204...409.
14   *              Attach that pin with ledcAttach / ledcSetup before it is read.
15   */
16
17  enum TelKind : uint8_t {
18    TEL_VEHICLE = 0,
19    TEL_ADC,
20    TEL_GPIO,
21    TEL_PWM,
22  };
23
24  enum TelVehicle : uint8_t {
```

```
25    TEL_TEMP = 0,
26    TEL_THROTTLE,
27    TEL_BATT,
28    TEL_STEER,
29    TEL_LIGHTS,
30    TEL_KNOB_L,
31    TEL_KNOB_R,
32    TEL_RY,
33  };
34
35  struct TelChannel {
36    TelKind kind;
37    TelVehicle vehicle;
38    int pin;
39    int ledcChannel;
40    int dutyMin;
41    int dutyMax;
42  };
43
44  using TelVehicleFn = uint8_t (*)(TelVehicle id);
45
46  inline uint8_t telemetryReadPwm(const TelChannel& ch) {
47    if (ch.dutyMax <= ch.dutyMin) return 0;
48    uint32_t duty = 0;
49  #if ESP_ARDUINO_VERSION_MAJOR >= 3
50    if (ch.pin >= 0) duty = ledcRead((uint8_t)ch.pin);
51  #else
52    if (ch.ledcChannel >= 0) duty = ledcRead((uint8_t)ch.ledcChannel);
53  #endif
54    long scaled = constrain((long)duty, ch.dutyMin, ch.dutyMax);
55    return (uint8_t)map(scaled, ch.dutyMin, ch.dutyMax, 0, 255);
56  }
57
58  inline uint8_t telemetryReadOne(const TelChannel& ch, TelVehicleFn vehicle) {
59    switch (ch.kind) {
60      case TEL_ADC:
61        if (ch.pin < 0) return 0;
62        return (uint8_t)map(constrain(analogRead(ch.pin), 0, 4095), 0, 4095, 0, 255);
63      case TEL_GPIO:
64        if (ch.pin < 0) return 0;
65        return digitalRead(ch.pin) ? 255 : 0;
66      case TEL_PWM:
67        return telemetryReadPwm(ch);
68      default:
69        return vehicle ? vehicle(ch.vehicle) : 0;
70    }
71  }
72
73  inline void telemetryReadAll(uint8_t out[8], const TelChannel table[8], TelVehicleFn vehicle) {
74    for (int i = 0; i < 8; i++) out[i] = telemetryReadOne(table[i], vehicle);
75  }
```